

Strings, branches and loops

Software Development: Programming and Algorithms

Dr Joseph Aylett-Bullock

FAQs

- Single and double quotation marks
- `print()` statements with different types
- Binary number storage

Week 1 FAQs notebook

Space for questions from last week.

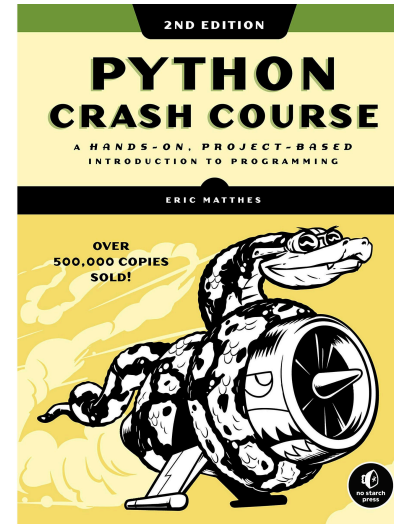
Overview and reading

Software Development: Programming and Algorithms

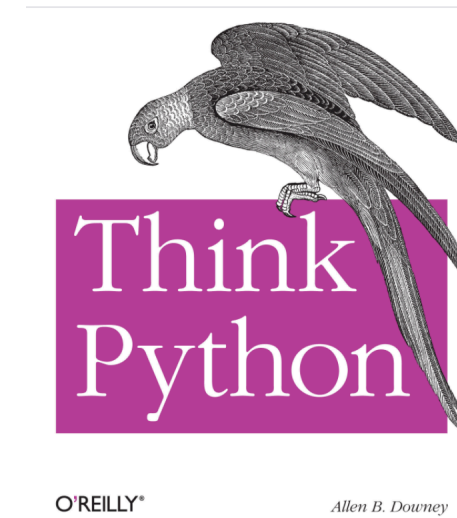
This week:

- Strings, methods and slices
- Conditional statements and indentation
- `while`, `for`, `range`, `break` and `continue`

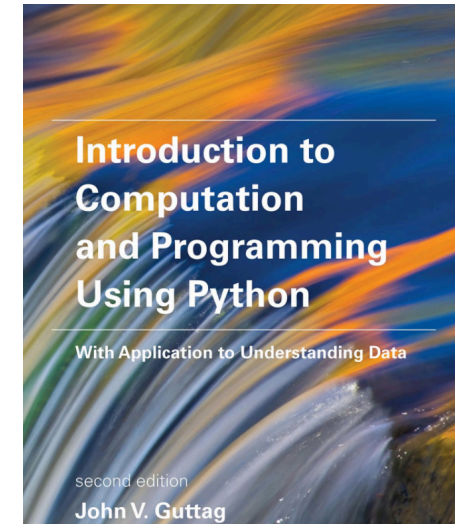
The robot example connects these ideas.



Matthes
Ch. 5, §7.1



Downey
§5.4, §§7.3–7.4
§§8.1, 8.5



Guttag
§§2.2, 2.4

Strings and operators

```
print("combine" + "these" + "together")
print("combine" + " " + "these" + " " + "together")
print("time: " + str(3) + " o'clock")
print("SPAM" * 3)
message = "I am a computer"
print(len(message))
print(type(len(message)))
```

Output

```
combinethesetogether
combine these together
time: 3 o'clock
SPAMSPAMSPAM
15
<class 'int'>
```

+ joins strings. ***** repeats a string an integer number of times.

len() gives the length. A string can contain letters, digits, spaces and Unicode characters.

Robot Lab

The robot begins **80 cm** from the wall. The detector supplies `raw`, a string such as `" d:080 "`.

`print("FORWARD")` moves the robot **20 cm**.

Open Robot Lab

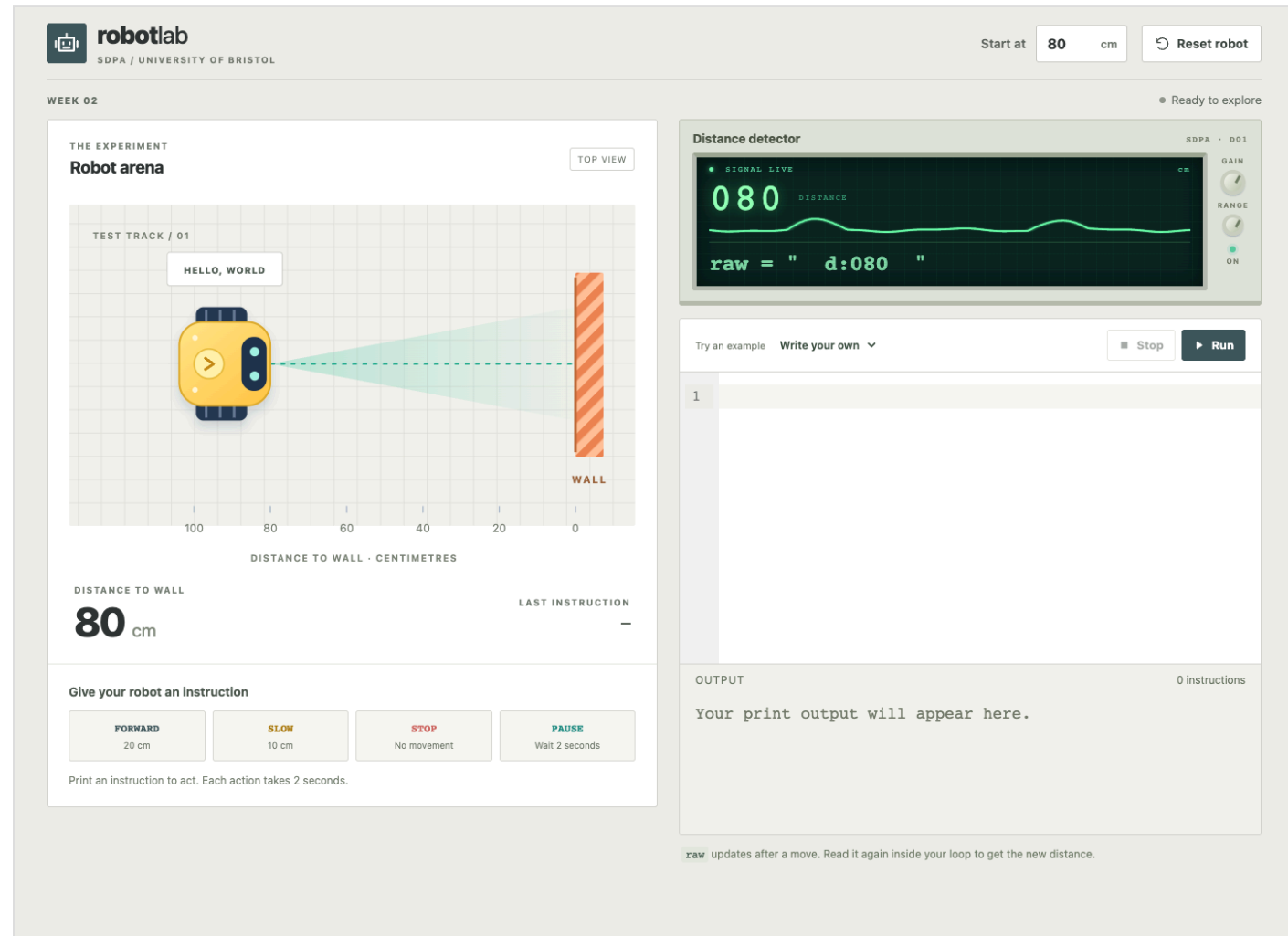
```
print(raw)
print("FORWARD")
print(raw)
```

Output

```
d:080
FORWARD
d:060
```

Final practice sensor: 60 cm; `raw = " d:060 "`

The Run button here uses a practice sensor. Open Robot Lab for the moving graphic. Each practice run starts at 80 cm.



The screenshot shows the Robot Lab interface. At the top, it says "robotlab SDPA / UNIVERSITY OF BRISTOL". On the right, there's a "Start at" field set to "80 cm" and a "Reset robot" button. Below this, it says "WEEK 02" and "Ready to explore".

The main area is titled "THE EXPERIMENT Robot arena" and "TOP VIEW". It shows a top-down view of a robot (a yellow circle with a blue square) on a grid. A green cone represents the robot's field of view, pointing towards a vertical orange and white striped bar labeled "WALL". A dashed line indicates the distance to the wall. Below the grid, it says "DISTANCE TO WALL - CENTIMETRES" and "80 cm".

Below the distance display, there's a section "Give your robot an instruction" with four buttons: "FORWARD 20 cm", "SLOW 10 cm", "STOP No movement", and "PAUSE Wait 2 seconds". Below these buttons, it says "Print an instruction to act. Each action takes 2 seconds."

On the right side, there's a "Distance detector" panel. It shows a green line graph of distance over time. The current distance is "080". Below the graph, it says "raw = " d:080 " ". There are also "GAIN", "RANGE", and "ON" controls.

Below the distance detector, there's a "Try an example" dropdown menu set to "Write your own". There are "Stop" and "Run" buttons. Below this, there's a "1" in a box, and an "OUTPUT" section that says "Your print output will appear here." and "0 instructions".

At the bottom, it says "raw updates after a move. Read it again inside your loop to get the new distance."

String methods

```
message = " ada Lovelace "  
print(message.upper())  
print(message.lower())  
print(message.title())  
print(repr(message.strip()))  
print(message.split(" "))  
print(" ".join(message.split()))
```

Output

```
ADA LOVELACE  
ada lovelace  
Ada Lovelace  
'ada Lovelace'  
['', 'ada', 'Lovelace', '']  
ada Lovelace
```

Methods can be chained: the next method acts on the value returned by the previous one. `strip()` removes outside whitespace; `join()` joins pieces of text.

Positions and slices

Character	P	Y	T	H	O	N
Index	0	1	2	3	4	5
Negative index	-6	-5	-4	-3	-2	-1

```
s = "PYTHON"
print(s[0])      # 'P'
print(s[-2])     # 'O'
print(s[0:3])    # 'PYT'
```

Output

```
P
O
PYT
```

s[start:stop:step] includes start, excludes stop and defaults to a step of **1**.

Slicing exercise

Drag tiles into the brackets. Predict before pressing **Slice**.

For `s = "PYTHON"`, what is `s[:3]`? **A** PYT **B** PYTH **C** YTH **D** error

s =

PYTHON

D:080

raw

Full screen

0

1

2

3

4

5

P

Y

T

H

O

N

-6

-5

-4

-3

-2

-1

s [

:

3

]

Slice

Clear

-6

-5

-4

-3

-2

-1

0

1

2

3

4

5

6

8

10

:

Or type inside []

:3

Drag tiles into a gap. Or tap a gap, then a tile. Tap a placed tile to remove it.

s[:3] = "PYT"

Selected indices: 0, 1, 2. Step 1; stop excluded.

Use the HTML deck for the interactive board.

Strings are immutable

An item assignment raises an error:

```
message = "ada Lovelace"  
message[0] = "A"
```

Output (intentional error)

```
TypeError: 'str' does not support item assignment  
on line 2
```

Reassignment uses a new string value:

```
message = "ada Lovelace"  
message = "A" + message[1:]  
print(message)
```

Output

```
Ada Lovelace
```

An item assignment cannot change the string. Reassignment binds the name to a new string value.

How do I remove the outside spaces and format the name properly in the fewest steps?

```
name = " ada loveLACE "  
name = name.strip().title()  
print(name)
```

Output

```
Ada Lovelace
```

Chaining the methods keeps the transformation in one expression.

The result is `Ada Lovelace`. Assign it if you want `name` to refer to the formatted value.

Robot Lab: extracting the distance

The detector supplies a **string** such as " d:080 ".

How do we extract the distance as an **int** and set it to **distance_cm**?

```
message = raw.strip()
distance_cm = int(message[2:])
print(distance_cm)
print(type(distance_cm))
```

Output

```
80
<class 'int'>
```

Final practice sensor: 80 cm; raw = " d:080 "

[Open Robot Lab](#)

One conditional block

VS Code/IDE

```
distance_cm = 80
if distance_cm > 20:
    print("FORWARD")
print("Reading checked")
```

Output

```
FORWARD
Reading checked
```

Robot Lab

```
message = raw.strip()
distance_cm = int(message[2:])
if distance_cm > 20:
    print("FORWARD")
print("Reading checked")
```

Output

```
FORWARD
Reading checked
```

Final practice sensor: 60 cm; raw = " d:060 "

The condition produces **True** or **False**. A **colon** starts the block. **Four spaces** show what belongs inside it.

Then Robot Lab · Keep hitting Run as the reading changes. At 20 cm, no FORWARD instruction runs.

A chain chooses at most one branch

```
raw = "  d:080  "  
message = raw.strip()  
distance_cm = int(message[2:])  
if distance_cm <= 20:  
    print("STOP")  
elif distance_cm <= 50:  
    print("SLOW")  
else:  
    print("FORWARD")  
print("Reading checked")
```

Output

```
FORWARD  
Reading checked
```

Python takes the **first true branch**. How would you rewrite this to print both **STOP** and **SLOW** when the distance is 20 cm?

Multiple **if** statements

```
distance_cm = 20
if distance_cm <= 20:
    print("STOP")
if distance_cm <= 50:
    print("SLOW")
print("Done")
```

Output

```
STOP
SLOW
Done
```

A STOP, Done **B** SLOW, Done **C** STOP, SLOW, Done

C. Each **if** starts a separate decision. Indentation decides which statements belong to a block.

Nested decisions

```
distance_cm = 80

if distance_cm <= 100:
    if distance_cm > 20:
        print("FORWARD")
    else:
        print("STOP")
else:
    print("PAUSE")
```

Output

FORWARD

`if distance_cm <= 100 and distance_cm > 20:` combines the tests for moving.

while loop

VS Code/IDE

```
distance_cm = 80
while distance_cm > 20:
    print("FORWARD")
    distance_cm -= 20
print("STOP")
```

Output

```
FORWARD
FORWARD
FORWARD
STOP
```

Robot Lab

```
message = raw.strip()
distance_cm = int(message[2:])
while distance_cm > 20:
    print("FORWARD")
    message = raw.strip()
    distance_cm = int(message[2:])
print("STOP")
```

Output

```
FORWARD
FORWARD
FORWARD
STOP
```

Final practice sensor: 20 cm; raw = " d:020 "

The IDE example changes a simulated distance. In Robot Lab, re-read the detector after moving.

Then Robot Lab · Reset to 80 cm, run, then try starting at 20 cm.

A stale reading

```
raw = " d:080 "  
message = raw.strip()  
distance_cm = int(message[2:])  
  
while distance_cm > 20:  
    print("FORWARD")
```

Output (intentional error)

```
FORWARD  
FORWARD  
FORWARD  
[Further repeated output omitted]  
RuntimeError: Run stopped after excessive output. Check the loop condition. on line 6
```

What happens next? **A** Stops at 20 **B** Keeps printing FORWARD **C** Does not print

B. `distance_cm` still contains `80`. In Robot Lab, read and convert the supplied `raw` after each move; do not overwrite it with a fixed string.

This deliberately nonterminating example can be interrupted with Stop. The slide runner also limits excessive output.

Repetition with keyboard input

A robot asks for a turn until you type `left`.

```
turn = input("Turn left or right? ").strip().lower()
while turn != "left":
    turn = input("Turn left or right? ").strip().lower()
print("Route confirmed")
```

Output

```
Turn left or right? right
Turn left or right? RIGHT
Turn left or right? alpaca
Turn left or right? left
Route confirmed
```

What happens for `right`, `RIGHT`, `alpaca`, then `left`?

for loops and range

Iterating through a string

```
route = "FFPFSFF"  
for instruction in route:  
    print(instruction)
```

Output

```
F  
F  
P  
F  
S  
F  
F
```

Each iteration visits the next character. No manual counter is needed.

Iterating through integers

```
for attempt in range(3):  
    print("Reading", attempt)  
for n in range(5, 40, 10):  
    print(n)  
for n in range(40, 5, -10):  
    print(n)
```

Output

```
Reading 0  
Reading 1  
Reading 2  
5  
15  
25  
35  
40  
30  
20  
10
```

Start included. Stop excluded. The defaults are **start 0** and **step 1**.

Reassigning the variable in the loop

```
x = 4
for i in range(x):
    print(i)
    x = 5
```

Output

```
0
1
2
3
```

A 0, 1, 2, 3 **B** 0, 1, 2, 3, 4 **C** Never finishes

A. Changing `x` does not rebuild the existing range for this loop.

break and continue

```
route = "FFPFSFF"
for instruction in route:
    if instruction == "S":
        print("STOP")
        break
    if instruction == "P":
        print("PAUSE")
        continue
    print("FORWARD")
```

Output

```
FORWARD
FORWARD
PAUSE
FORWARD
STOP
```

break exits this loop. **continue** skips to its next iteration.

Robot Lab: a route string

Reset to **80 cm**. Build a **for** loop for **"FFPFSFF"**.

F means FORWARD. **S** means STOP and **break**. **P** means PAUSE and **continue**.

Final distance: **A** 0 **B** 20 **C** 40 **D** 60 cm

B. Three moves, one pause, then STOP. The final two **F**s are skipped.

Open Robot Lab

```
message = raw.strip()
distance_cm = int(message[2:])
for instruction in "FFPFSFF":
    if instruction == "S":
        print("STOP")
        break
    if instruction == "P":
        print("PAUSE")
        continue
    print("FORWARD")
message = raw.strip()
distance_cm = int(message[2:])
```

Output

```
FORWARD
FORWARD
PAUSE
FORWARD
STOP
```

Final practice sensor: 20 cm; raw = " d:020 "

break in the innermost loop

```
for trial in range(2):  
    for instruction in "FSF":  
        if instruction == "S":  
            break  
    print("Trial finished", trial)
```

Output

```
Trial finished 0  
Trial finished 1
```

How many times does the final `print` run?

Twice. The outer loop continues after the inner loop ends.

References

- **Python Crash Course**, 2nd ed., Eric Matthes: chapter 5 and §7.1.
- **Think Python**, Allen B. Downey: §5.4, §§7.3–7.4, §§8.1 and 8.5, using the course edition.
- **Introduction to Computation and Programming Using Python**, 2nd ed., John V. Guttag: §§2.2 and 2.4.

Week 1 FAQs notebook

The following two slides collect additional examples for reference.

Reference: string methods

```
s = "  ada LOVELACE  "
print(s.upper())
print(s.lower())
print(s.title())
print(repr(s.lstrip()))
print(repr(s.rstrip()))
print(repr(s.strip()))
print(len(s))
```

Output

```
ADA LOVELACE
ada lovelace
Ada Lovelace
'ada LOVELACE '
'  ada LOVELACE'
'ada LOVELACE'
16
```

```
s = "PYTHON"
print(s[:3])
print(s[3:])
print(s[::2])
print(s[::-1])
print(str(80), int("080"))
```

Output

```
PYT
HON
PTO
NOHTYP
80 80
```

Strings are immutable. Methods return values; assignment chooses what a name refers to. `len()` counts Unicode code points, which need not equal the number of visible symbols.

Reference: looping approaches

Characters directly

```
s = "FFPFSFF"
for ch in s:
    print(ch)
```

Output

```
F
F
P
F
S
F
F
```

Positions from `range`

```
s = "FFPFSFF"
for i in range(len(s)):
    print(i, s[i])
```

Output

```
0 F
1 F
2 P
3 F
4 S
5 F
6 F
```

A `while` counter

```
s = "FFPFSFF"
i = 0
while i < len(s):
    print(i, s[i])
    i += 1
```

Output

```
0 F
1 F
2 P
3 F
4 S
5 F
6 F
```

Direct iteration is usually clearest when only the character matters. A `while` counter needs an explicit update.